Mental Health Signal Analysis System:

Evolution from Disorder Detection to Multi-Task NLP Pipeline

A PROJECT REPORT

Submitted By

Avi Ambastha, Autonomy Roll No.-12622001044, Reg. No.- 221260110274
Ekam Bitt, Autonomy Roll No. 12622001064, Reg. No. 221260110294
Abhishek Prasad, Autonomy Roll No. 12622001004, Reg. No. 221260110233
Ayush Chhajer, Autonomy Roll No. 12622001049, Reg. No. 221260110279

Under the Guidance of

Prof. Amitabha Acharya
Dept. of Computer Science and Engineering
Heritage Institute of Technology, Kolkata

In partial fulfillment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING
HERITAGE INSTITUTE OF TECHNOLOGY,
KOLKATA

An autonomous institute under
Maulana Abul Kalam Azad University of Technology

May, 2026

ACKNOWLEDGEMENT

We would like to thank **Prof. (Dr.) Basab Choudhury**, Principal Heritage Institute of Technology, for providing us with all the necessary facilities to make our project work successful.

We would like to thank our Head of the Department, **Prof. (Dr.) Subhashis Majumder** for his kind assistance as and when required.

We will be thankful to **Prof. Amitabha Acharya**, our project coordinator, for constantly supporting and guiding us for giving us invaluable insights. His guidance and encouragement motivated us to achieve our goal and impetus to excel.

We thank our faculty members and laboratory assistants at the Heritage Institute of Technology for playing a pivotal and decisive role during the project's development. Last but not least, we thank all our friends for their cooperation and encouragement that they bestowed on us.

Finally, we are grateful to the online communities and researchers who have made public datasets, model checkpoints, and research papers freely accessible, thereby enabling applied work such as ours. This project drew on a wide body of knowledge ranging from transformer-based natural language processing to privacy-preserving system design and responsible AI development.

Avi Ambastha

Ekam Bitt

Abhishek Prasad

Ayush Chhajer

HERITAGE INSTITUTE OF TECHNOLOGY, KOLKATA
An autonomous institute under
Maulana Abdul Kalam Azad University of Technology

BONAFIDE CERTIFICATE

Certified that this project report on "Mental Health Signal Analysis System: Evolution from Disorder Detection to Multi-Task NLP Pipeline" is the bonafide work of "Avi Ambastha", "Ekam Bitt", "Abhishek Prasad", and "Ayush Chhajer who carried out the project.

.

SIGNATURE	**SIGNATURE**
Prof. (Dr.) Subhashis Majumder	Prof. Amitabha Acharya
Professor and HoD	**PROJECT GUIDE**
Dean, UG Programme	
Computer Science and Engineering,	Computer Science and Engineering

SIGNATURE
EXTERNAL EXAMINER

ABSTRACT

The proliferation of social media platforms has created vast repositories of user-generated content in which people routinely express emotional distress and mental-health-related language signals. This project presents Mental Wellbeing Guard, a dual-mode Chrome browser extension and local web hub designed to analyse mental-health-related linguistic signals in social media comments and provide awareness-oriented, non-diagnostic wellbeing support.

The system is built around a fine-tuned RoBERTa sequence classifier exported to a quantized ONNX artifact, achieving approximately 75 per cent reduction in model storage from 499 MB to 125 MB without retraining. The underlying model recorded 85.69 per cent accuracy and a macro F1-score of 0.8601 on a seven-class classification task.

Mental Wellbeing Guard operates in two modes. Cloud Mode routes inference to a hosted Hugging Face Spaces FastAPI service, enabling zero-setup usability. Local Mode routes inference to a Dockerized Flask backend, persists wellbeing history in a local SQLite database, and provides a full-featured web hub with weekly dashboard, emotional-diet breakdown, self-check interface, and browsing-session tracking. The extension supports passive monitoring and manual thread scanning on YouTube, Reddit, X, and Twitter, and includes Smart Shield content blurring, rabbit-hole nudges, a five-minute breather mode, and draft-time support prompts with locale-aware crisis resources.

Keywords: mental health NLP, RoBERTa, ONNX Runtime, Chrome Manifest V3, Hugging Face Spaces, Docker, Flask, FastAPI, social media text classification, digital wellbeing, dual-mode inference, responsible AI.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ADHD	Attention Deficit Hyperactivity Disorder
AI	Artificial Intelligence
API	Application Programming Interface
BPD	Borderline Personality Disorder
BERT	Bidirectional Encoder Representations from Transformers
CI/CD	Continuous Integration / Continuous Delivery
CORS	Cross-Origin Resource Sharing
DOM	Document Object Model
GPU	Graphics Processing Unit
HF	Hugging Face
INT8	8-bit Integer Quantization
ML	Machine Learning
MV3	Manifest Version 3
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
PTSD	Post-Traumatic Stress Disorder
REST	Representational State Transfer
RoBERTa	Robustly Optimized BERT Pretraining Approach
SQLite	Self-Contained Relational Database Engine
TF-IDF	Term Frequency-Inverse Document Frequency
WSGI	Web Server Gateway Interface

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1

1.1 Background and Motivation 1

1.2 The Mental Wellbeing Guard Solution 1

1.3 Project Evolution: V1 to V2.0.. 2

1.4 Objectives and Design Principles 2

CHAPTER 2: SCOPE OF THE PROJECT 3

2.1 Supported Platforms.. 3

2.2 Inference Modes.. 3

2.3 Model Label Scope and Ethical Boundaries 4

CHAPTER 3: LITERATURE SURVEY 5

3.1 Transformer Foundations 5

3.2 Mental Health NLP on Social Media 5

3.3 Efficient Deployment and ONNX Optimization 5

3.4 Responsible AI and Gap Analysis 6

CHAPTER 4: ML PIPELINE – DATA COLLECTION, ANNOTATION, MODEL TRAINING AND DEPLOYMENT 7

4.1 Problem Framing and Dataset Design 7

4.1.1 Label Taxonomy 7

4.1.2 Dataset Schema 8

4.2 Data Collection and LLM Annotation Pipeline 9

4.2.1 LLM Annotator Design 9

4.2.2 Data Quality Controls 10

4.3 Model Architecture 11

4.3.1 MentalHealthModel Architecture 11

4.3.2 Loss Function: Focal BCE 12

4.4 Training Pipeline 13

4.4.1 Stage 1 — Domain-Adaptive Pre-Training (DAPT) 13

4.4.2 Stage 2 — Supervised Teacher Fine-Tuning 14

4.4.3 Stage 3 — Teacher-to-Student Knowledge Distillation.............................. 16

4.5 Post-Training: Calibration and Threshold Optimisation 17

4.5.1 Temperature Scaling Calibration 17

4.5.2 Per-Label Threshold Optimisation 17
4.6 ONNX Export and INT8 Quantization 18
4.7 Inference Pipeline 19
4.8 Ablation Study Design 20
CHAPTER 5: TECHNOLOGY STACK 22
5.1 Browser Extension and Backend Technologies 22
5.2 Inference Runtime, Database, and DevOps 23
CHAPTER 6: SYSTEM ARCHITECTURE AND DESIGN 24
6.1 Architectural Overview 24
6.2 Cloud and Local Mode Data Flows 24
6.3 Extension Module Architecture 25
6.4 Local Web Hub and SQLite Store 26
6.5 API Contracts 26
6.6 Risk Scoring and Wellbeing Metrics 27
6.7 Key Implementation Constants 28
CHAPTER 7: REQUIREMENTS 31
CHAPTER 8: IMPLEMENTATION DETAILS 33
8.1 Extension Implementation 33
8.2 Hugging Face Space API 34
8.3 Local Flask Backend 35
8.4 Docker Deployment 35
8.5 Implemented Features Summary 35
CHAPTER 9: RESULTS AND DISCUSSION 38
9.1 Historical Model Evaluation (V1 Phase) 38
9.2 ONNX Quantisation Results 38
9.3 V1.0 to V2.0 Product Improvements 39
9.4 Discussion 40
CHAPTER 10: ETHICAL CONSIDERATIONS AND PRIVACY 41
10.1 Signal Analysis vs. Clinical Diagnosis 41
10.2 Privacy Architecture 41
10.3 Data Ethics, User Autonomy, and Responsible AI Principles 41
CHAPTER 11:TESTING AND VALIDATION 43
11.1 Backend Automated Tests 43

11.2 CI Workflow Validation 44
11.3 Manual Validation Summary.. 45
CHAPTER 12: LIMITATIONS.. 46
CHAPTER 13: FUTURE SCOPE 48
CHAPTER 14: CONCLUSION.. 49
14.1 Project Summary.. 49
14.2 Technical and Responsible AI Contributions 49
References 50

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Social media platforms such as YouTube, Reddit, X, and Twitter collectively host hundreds of millions of posts and comments every day. Within this volume, a substantial proportion contains language reflecting emotional distress and mental-health-related concerns. Research has consistently demonstrated that NLP can detect meaningful language signals in such text: Coppersmith et al. (2014) showed quantifiable linguistic patterns correlating with mental health disclosures on Twitter; Tadesse et al. (2019) demonstrated depression-related post identification on Reddit with competitive accuracy. The emergence of large pre-trained transformers — BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019) — substantially raised the capability ceiling for such classification tasks.

Despite this progress, deploying mental health NLP systems carries ethical risk. Proxy labels derived from community membership rather than clinician assessment, short context-free posts, and diverse user language introduce noise and uncertainty into classifier outputs. A system presenting classifier results as clinical diagnoses risks harming users. Responsible AI practice in this domain requires framing outputs as probabilistic signals, disclosing privacy tradeoffs, and incorporating support resources directing users to professional help when appropriate.

1.2 The Mental Wellbeing Guard Solution

Mental Wellbeing Guard is a dual-mode Chrome browser extension and local web hub that analyses mental-health-related linguistic signals in social media comments and converts these signals into interpretable summaries, content-safety actions, and wellbeing dashboard metrics. It is designed for personal awareness and digital wellbeing support, not clinical diagnosis. The inference model is a fine-tuned RoBERTa sequence classifier exported to an INT8-quantized ONNX artifact, served through ONNX Runtime on CPU in two deployment paths: a zero-setup Hugging Face Spaces cloud API, and a privacy-first Dockerized Flask local backend with SQLite history and a web hub.

1.3 Project Evolution: V1 to V2.0

The project originated as Detection-of-Mental-Disorders-Extension, a prototype pairing a manual comment scan interface with a local Python backend. V1 established core technical feasibility and produced baseline evaluation results: 85.69 per cent accuracy and a macro F1 of 0.8601 on the seven-class task. V1 had two key limitations: Docker setup required for all users (friction for non-technical users), and disorder-detection framing (ethical risk). Version 2.0 addressed both: the product was renamed and reframed around signal analysis; Cloud Mode was introduced as the zero-setup default; Local Mode was enhanced with SQLite wellbeing history and a web hub; inference was migrated to ONNX Runtime; and the extension gained passive monitoring, Smart Shield blurring, rabbit-hole nudges, breather mode, draft-time support prompts, and a full GitHub Actions CI pipeline.

1.4 Objectives and Design Principles

The primary objectives are: (1) browser-based comment extraction from four platforms; (2) ONNX Runtime seven-class inference; (3) zero-setup Cloud Mode via Hugging Face Spaces; (4) privacy-first Local Mode with full local inference and SQLite persistence; (5) wellbeing dashboard, self-check, and session history; (6) Smart Shield, nudges, breather mode, and locale-aware draft-time prompts; and (7) consistent non-diagnostic framing throughout. Three design principles guided all decisions: privacy by design (stateless cloud inference separated from stateful local tracking, with the tradeoff visible in the UI); responsible framing (outputs are signals, not diagnoses); and user autonomy (all interventions are reversible overlays).

CHAPTER 2: SCOPE OF THE PROJECT

2.1 Supported Platforms

Platform	Target Element	Extraction Mode	Notes
YouTube	ytd-comment-thread-renderer	Manual + Passive	Requires comment section scrolled into view
Reddit	shreddit-comment	Manual + Passive	New Reddit UI only
X (Twitter)	article[data-testid]	Manual + Passive	Timeline and thread views
Twitter	article[data-testid]	Manual + Passive	Legacy twitter.com domain

Table 2.1: Supported Platforms and Extraction Scope

2.2 Inference Modes

Property	Cloud Mode	Local Mode
Default	Yes	No (opt-in)
Setup required	None	Docker Compose
Inference endpoint	Hugging Face Space	http://localhost:8000
Data off-device	Comment text sent to HF Space	No data leaves machine
Dashboard history	Not available	Available (SQLite)
Self-check feature	Not available	Available
Web hub	Not available	Available at localhost:8000

Table 2.2: Inference Mode Comparison: Cloud Mode vs. Local Mode

2.3 Model Label Scope and Ethical Boundaries

The classifier assigns probability scores across seven label categories derived from the fine-tuning dataset: ADHD, Anxiety, Autism, BPD, Depression, PTSD, and Normal. These are community-derived proxy labels, not clinical diagnoses. Out of scope: clinical diagnosis; user profiling; content removal or blocking; access to browser authentication data; and any form of medical advice generation. All interventions are awareness-only overlays that preserve full user control.

CHAPTER 3: LITERATURE SURVEY

3.1 Transformer Foundations

Vaswani et al. (2017) introduced the Transformer architecture, establishing self-attention as the dominant mechanism for sequence modelling. Devlin et al. (2019) applied bidirectional pre-training to produce BERT, which set new NLP benchmarks. Liu et al. (2019) extended BERT's pre-training in RoBERTa, demonstrating robust representations for downstream classification. Sanh et al. (2019) applied knowledge distillation to produce DistilBERT — a principled compression approach extended to INT8 quantization in this project.

3.2 Mental Health NLP on Social Media

Coppersmith et al. (2014) quantified mental health language signals on Twitter, establishing proxy labels from user disclosures as viable for large-scale analysis. Tadesse et al. (2019) achieved strong depression detection on Reddit using deep learning. Ji et al. (2021) reviewed suicidal ideation detection methods, emphasising the gap between proxy-label accuracy and clinical validity. Chancellor and De Choudhury (2020) critically assessed predictive techniques, arguing that proxy labels capture community-level language patterns but should not be interpreted as individual clinical status — directly motivating this project's responsible framing.

3.3 Efficient Deployment and ONNX Optimization

ONNX provides a standardised intermediate representation enabling framework-independent deployment. Microsoft's ONNX Runtime supports INT8 quantization, reducing model storage and accelerating CPU inference without retraining. This project exports the fine-tuned RoBERTa checkpoint to INT8 ONNX, achieving a 75 per cent size reduction (499 MB to 125 MB). Chrome Manifest V3's service-worker model and FastAPI's async execution model informed the extension and cloud API architecture respectively.

3.4 Responsible AI and Gap Analysis

Bender et al. (2021) argued that large language models encode societal biases requiring transparency about training data, limitations, and intended use. Conway and O'Connor (2016) identified consent, privacy, and data security as primary concerns in social media mental health research. The gap analysis identifies that existing work focuses on model development with limited attention to end-to-end consumer deployment; browser extension implementations with privacy-first options are rare; and the combination of dual-mode deployment, real-time browser integration, and user-facing wellbeing features in a single system has not been demonstrated in the literature.

CHAPTER 4: ML PIPELINE – DATA COLLECTION, ANNOTATION, MODEL TRAINING AND DEPLOYMENT

4.1 Problem Framing and Dataset Design

The V1 prototype framed the task as multi-class single-label disorder detection using subreddit membership as the sole supervision signal: a post sourced from r/depression received the label Depression regardless of its actual content. While computationally convenient, this weak-supervision scheme conflates community membership with individual language signal, introducing label noise and raising ethical concerns about diagnostic framing. The V2.0 system reframes the task as multi-label mental-health signal analysis: a post can simultaneously carry signals of depressive affect, anxious affect, and trauma stress, with an accompanying ordinal severity estimate. This shift required a full dataset redesign covering label taxonomy, schema, annotation strategy, and split construction.

4.1.1 Label Taxonomy

Eight mutually non-exclusive signal classes were defined after reviewing existing mental health NLP literature and aligning with responsible framing constraints. Each label describes an observable language pattern rather than a clinical category:

Signal Label	Observable Linguistic Pattern	Ethical Framing Note
attention_dysregulation	ADHD-adjacent: focus difficulties, impulsivity, disorganisation language	Signal, not diagnosis of ADHD
anxious_affect	Panic, excessive worry, avoidance, hypervigilance	Signal of anxious language, not GAD/panic disorder

autistic_trait_discussion	Sensory sensitivity, social difficulty, stimming descriptions	Self-reported trait discussion, not diagnostic claim
emotional_instability	Intense mood swings, splitting, rage/abandonment language	BPD-adjacent affect signal only
depressive_affect	Hopelessness, anhedonia, low energy, self-worth language	Signal of depressive language, not MDD
trauma_stress	Flashbacks, hyperarousal, abuse or PTSD-adjacent references	Trauma-language signal, not PTSD diagnosis
crisis_self_harm	Explicit self-harm, suicidal ideation, or crisis-state language	Treated with highest classifier weight and UI priority
no_clear_signal	Neutral or off-topic content with no detectable mental-health signal	Majority class; used as negative in BCE training

Table 4.1: Signal Label Taxonomy and Ethical Framing

A secondary ordinal target, severity, accompanies each example with four levels: 0 = none, 1 = mild, 2 = moderate, 3 = high. Severity is modelled as a classification head rather than regression to avoid implying cardinal scale properties that the underlying language patterns do not support.

4.1.2 Dataset Schema

All training examples follow a unified schema regardless of source platform. The schema was designed to support multi-source training, provenance tracking, and reproducible split construction:

Field	Type	Description
text	string	Raw post or comment text; no anonymisation beyond what the platform applies
label_signals	list[string]	One or more signal labels from the taxonomy; multi-label, not mutually exclusive
severity	int {0,1,2,3}	Ordinal severity estimate assigned during annotation
annotator_confidence	float [0.0, 1.0]	LLM annotator self-reported confidence; used as a sample weight during training
split	string enum	train | val | test | cross_platform_holdout

Table 4.2: Unified Dataset Schema

4.2 Data Collection and LLM Annotation Pipeline

The raw dataset comprises approximately 10,000 Reddit posts sourced from mental-health-adjacent subreddits and collected into reddit_test.csv. This corpus was used as a weak-label bootstrap: posts carry an existing numeric label derived from subreddit membership, but this label is used only for stratification during sampling and is not propagated into the V2.0 training targets. The definitive signal labels are assigned through a dedicated LLM annotation pipeline developed in the annotate_data module of the training repository.

4.2.1 LLM Annotator Design

The annotation pipeline (llm_annotator.py) uses a locally-hosted medical-domain language model — MedGemma 1.5 4B (**dcarrascosa/medgemma-1.5-4b-it:Q4_K_M**) served via Ollama — to generate structured multi-label annotations for each post. MedGemma is selected over a general-purpose LLM because its pre-training incorporates biomedical and clinical text, producing better calibrated mental health signal recognition. The Q4_K_M quantization enables the model to run on consumer hardware without GPU, making the annotation pipeline reproducible without cloud compute.

The annotator implements the following processing logic: (1) the CSV is loaded and the text column is auto-detected by probing for common field names (post, text, body); (2) posts shorter than 15 words or 45 characters are filtered out as insufficient context for reliable annotation; (3) each qualifying post is submitted to the LLM with a structured prompt specifying the eight-label taxonomy, the severity scale, and a confidence range; (4) the LLM returns a JSON-structured response containing label_signals (list), severity (int), and annotator_confidence (float); (5) each result is written as a line to a JSONL output file with full provenance fields preserved; (6) on LLM failure for any row, a safe fallback of no_clear_signal, severity 0, confidence 0.0 is written, and the row is excluded from stratified sampling in downstream steps.

This annotation strategy produces training labels grounded in actual post content and in a medically-aware model's interpretation of that content — a substantial improvement over the V1 approach of treating subreddit membership as a proxy for individual post label. The JSONL output is the handoff artefact passed to the model training team for dataset construction.

4.2.2 Data Quality Controls

Three quality controls are applied before the annotated corpus enters model training. First, split construction isolates examples at the author/thread level: all posts from a given thread_id are assigned to the same split, preventing the model from memorising thread-level context rather than generalising linguistic patterns. Second, a cross-platform holdout split is constructed by excluding all examples from one source platform during training and using that platform exclusively for out-of-domain evaluation — this directly tests cross-platform generalisation. Third, label frequency and severity distribution checks are run as automated data tests: if any signal class falls below a minimum support threshold after splitting, an alert is raised and the split seed is adjusted before training proceeds.

4.3 Model Architecture

The V2.0 system uses a teacher-student architecture: a full RoBERTa-base model is trained as the high-accuracy research model (teacher), and its knowledge is distilled into a DistilRoBERTa-base model (student) that is exported to ONNX for production serving. Both models share the same custom multi-task architecture, MentalHealthModel, implemented in src/model.py.

4.3.1 MentalHealthModel Architecture

The MentalHealthModel class wraps a HuggingFace AutoModel encoder and attaches three task-specific linear heads on top of the CLS token representation. The architecture is intentionally shallow above the transformer to preserve the pre-trained representations while allowing task-specific adaptation:

Component	Input	Output	Notes
AutoModel encoder	input_ids, attention_mask	last_hidden_state [B, L, H]	RoBERTa-base (H=768) or DistilRoBERTa-base (H=768)
CLS pooling	last_hidden_state	pooled [B, H]	Index 0 of sequence output; no mean pooling
Dropout (p=0.1)	pooled	pooled (regularised)	Applied before all heads
signal_head (Linear)	pooled [B, 768]	signal_logits [B, 8]	Multi-label; trained with Focal BCE loss
severity_head (Linear)	pooled [B, 768]	severity_logits [B, 4]	Ordinal classification; trained with cross-entropy

platform_head (Linear)	pooled [B, 768]	platform_logits [B, P]	Optional; used for domain robustness analysis only

Table 4.3: MentalHealthModel Component Summary

The signal head produces eight independent logits which are passed through a sigmoid activation at inference time to obtain per-label probabilities. Labels are predicted as positive when their probability exceeds a per-label threshold, defaulting to 0.5 and refined through threshold optimisation on the validation set. The severity head produces four logits corresponding to the ordinal classes; the predicted severity is the argmax. The platform head, when active, produces logits for source platform classification and is used during training to encourage the encoder to learn platform-invariant representations.

4.3.2 Loss Function: Focal BCE

Standard binary cross-entropy loss is poorly suited to the mental health signal dataset because the label distribution is severely imbalanced: crisis_self_harm and autistic_trait_discussion are rare relative to depressive_affect and no_clear_signal. The Focal BCE loss (FocalBCELoss in src/losses.py) addresses this by down-weighting easy negative examples. Given a binary cross-entropy loss BCE(y, ŷ) for a single label, the focal variant modulates it by a factor $(1 - p_t)^\gamma$ where p_t is the probability assigned to the correct class and γ is a focusing parameter set to 2.0:

$$L_{focal} = \alpha \cdot (1 - p_t)^\gamma \cdot BCE(y, \hat{y})$$

The effect is that examples the model already classifies correctly (high p_t) contribute small gradients, while hard or misclassified examples (low p_t) drive the bulk of the weight updates. The combined training objective weights the signal loss as the primary task and the severity loss as a regularising auxiliary: $L_{total} = L_{focal(signals)} + 0.3 \cdot L_{CE(severity)}$.

4.4 Training Pipeline

The training pipeline proceeds through three sequential stages: domain-adaptive pre-training (DAPT), supervised teacher fine-tuning, and teacher-to-student distillation. Each stage uses a dedicated script in src/, ensuring clean separation of concerns and reproducibility via config files.

4.4.1 Stage 1 — Domain-Adaptive Pre-Training (DAPT)

RoBERTa-base is pre-trained on general English web text and lacks exposure to the idiomatic, abbreviated, and emotionally charged language that characterises social media mental health posts. Domain-Adaptive Pre-Training addresses this by continuing the masked language modelling (MLM) objective on unlabeled social media text (data/unlabeled.csv) before any task-specific fine-tuning.

The DAPT stage (src/train_dapt.py) loads the base RoBERTa-base checkpoint for masked language modelling using HuggingFace's AutoModelForMaskedLM. A DataCollatorForLanguageModeling applies random 15 per cent token masking inline during training. Training runs for five epochs with a learning rate of 5×10^{-5}, weight decay of 0.01, and FP16 mixed precision. The adapted checkpoint is saved to checkpoints/dapt-final and used as the initialisation point for teacher fine-tuning rather than the original RoBERTa-base weights. This domain shift step consistently improves downstream classification performance for social-media NLP tasks by narrowing the distributional gap between pre-training corpus and deployment domain.

Hyperparameter	Value	Rationale
Base model	FacebookAI/roberta-base	Strong general-purpose English encoder; byte-level BPE handles social media spelling variation
MLM probability	0.15	Standard BERT/RoBERTa masking rate

Learning rate	5×10^{-5}	Higher than fine-tuning LR; MLM is a pre-training objective, not task-specific
Epochs	5	Sufficient domain shift without catastrophic forgetting on low-resource unlabeled corpus
Batch size	8	Memory-constrained; gradient accumulation can extend effective batch size
Mixed precision	FP16	GPU memory reduction; no quality impact on MLM

Table 4.4: DAPT Hyperparameters

4.4.2 Stage 2 — Supervised Teacher Fine-Tuning

The DAPT checkpoint is loaded as the encoder backbone for MentalHealthModel and fine-tuned on the annotated dataset (data/labeled.csv). The training script (src/train_teacher.py) implements a standard supervised training loop with several research-grade additions: mixed-task loss weighting, gradient norm clipping, learning rate warm-up, and patience-based early stopping.

The dataset is split 90/10 into training and validation sets using a fixed random seed of 42. The MentalHealthDataset class tokenizes each post with RoBERTa's byte-level BPE tokenizer, truncating to a maximum of 512 tokens and padding to the maximum sequence length in each batch. Each example returns input_ids, attention_mask, a float tensor of signal labels for the multi-label head, and an integer severity class for the ordinal head.

Hyperparameter	Value	Rationale
Base checkpoint	checkpoints/dapt-final	Domain-adapted weights; better than raw roberta-base for social media text

Learning rate	2×10^{-5}	Standard fine-tuning LR for RoBERTa; higher LRs destabilise attention layers
Warmup ratio	0.1	10% of total steps as linear warm-up; prevents early large gradient updates
Batch size	8	Per-device; effective 8 per GPU step
Epochs	5 (max)	Early stopping on validation macro-F1 with patience=2 prevents overfitting
Weight decay	0.01	AdamW regularisation; standard for transformer fine-tuning
Gradient clip	1.0	Norm clipping prevents gradient explosion in early epochs
Signal loss	Focal BCE ($\gamma = 2.0, \alpha = 1.0$)	Down-weights easy negatives; addresses class imbalance across 8 labels
Severity loss	Cross-entropy	Standard ordinal classification; weighted 0.3× as auxiliary task
Max sequence len	512 tokens	Full RoBERTa context; long Reddit posts benefit from extended context

Table 4.5: Teacher Fine-Tuning Hyperparameters

The early stopping mechanism monitors validation macro-F1 at the end of each epoch. If macro-F1 fails to improve for two consecutive epochs, training terminates and the best checkpoint is saved to checkpoints/best_teacher.pt. The best model from the V1 training phase achieved 85.69 per cent accuracy and a macro F1 of 0.8601 on the seven-class proxy-label dataset, establishing the performance baseline that V2.0 aims to surpass on the richer, multi-label annotated dataset.

4.4.3 Stage 3 — Teacher-to-Student Knowledge Distillation

Deploying the full 125M-parameter RoBERTa-base teacher in a browser extension local backend would impose unacceptable memory and latency costs. Knowledge distillation (Hinton et al., 2015) compresses the teacher's learned representations into a smaller student model — DistilRoBERTa-base — while retaining a high fraction of teacher performance. The distillation script (src/distill_student.py) implements a combined objective of soft-target KL divergence loss and hard-label BCE loss.

During distillation training, the frozen teacher generates soft probability distributions over the eight signal classes for each training batch. These soft targets encode class similarity information unavailable in hard binary labels: if the teacher assigns 0.6 probability to depressive_affect and 0.3 to trauma_stress for a given post, the student is trained to reproduce this distribution rather than simply predict the hard label. The soft distributions are computed using a temperature T = 3.0 applied to both teacher and student logits before the softmax, which flattens the distribution and makes the class similarity information more prominent. The combined distillation loss is:

$$L_{distill} = \alpha \cdot T^2 \cdot KL(soft_{student} \parallel soft_{teacher}) + (1 - \alpha) \cdot BCE(student_{logits}, hard_{labels})$$

where $\alpha = 0.7$ weights the soft-target KL term as dominant and T^2 rescales the KL gradient to match the magnitude of the hard-label BCE gradient. The student is trained with AdamW at a learning rate of 2×10^{-5}. The resulting DistilRoBERTa student model is smaller, faster, and suitable for CPU inference in Local Mode.

Parameter	Value	Effect
Student model	distilroberta-base	40% fewer parameters than RoBERTa-base; retains 95%+ task performance
Temperature (T)	3.0	Softens teacher distributions; exposes inter-class similarity to student

Alpha (α)	0.7	70% weight on soft KL loss; 30% on hard BCE loss
KL scaling	$T^2 = 9.0$	Compensates for gradient magnitude reduction from temperature scaling
Distillation loss	KL divergence	Measures divergence between student and teacher soft distributions
Hard loss	Binary cross-entropy	Ensures student also optimises directly on ground-truth labels

Table 4.6: Knowledge Distillation Parameters

4.5 Post-Training: Calibration and Threshold Optimisation

Two post-training steps improve the reliability of the deployed student model's probability outputs before ONNX export.

4.5.1 Temperature Scaling Calibration

Transformer classifiers are frequently overconfident: they assign high probabilities to predictions even when the true uncertainty is substantial. Temperature scaling (src/calibrate.py) is a single-parameter post-hoc calibration method that divides all logits by a learned scalar temperature T before the sigmoid activation. The optimal temperature is found by minimising the binary cross-entropy of the calibrated predictions on the held-out validation set using scipy.optimize.minimize with bounds [0.05, 10.0]. A temperature greater than 1.0 spreads the probability mass, reducing overconfidence; a temperature below 1.0 sharpens it. Calibrated probabilities are more meaningful as confidence scores in the risk score formula used by the extension's wellbeing dashboard.

4.5.2 Per-Label Threshold Optimisation

Because the eight signal classes have different base rates and different costs of false positives versus false negatives, a single global threshold of 0.5 applied to all sigmoid outputs is suboptimal. The optimize_thresholds function (src/metrics.py) performs a grid search over thresholds in [0.1, 0.9] with step 0.05 for each label independently, selecting the threshold that maximises per-label F1 on the validation set. The resulting per-label threshold vector replaces the default 0.5 in the inference module, improving macro-F1 at deployment time without any additional training.

4.6 ONNX Export and INT8 Quantization

The calibrated student model is exported to the ONNX intermediate representation using HuggingFace Optimum's ORTModelForSequenceClassification (src/export_onnx.py), which traces the PyTorch computation graph and serializes it as a portable ONNX artifact. The exported model and its tokenizer are saved to onnx/student/.

The ONNX model is subsequently quantized to INT8 precision using ONNX Runtime's dynamic quantization API (src/quantize.py). Dynamic quantization converts weight tensors from FP32 to INT8 at export time while computing activation quantization scales at runtime, requiring no calibration data. This produces the model-int8.onnx artifact that is uploaded to the ekam28/emotion-detector-onnx Hugging Face repository and served by both the Cloud Mode FastAPI endpoint and the Local Mode Flask backend via ONNX Runtime.

Artefact	Size	Inference Backend	Deployment Target
roberta-base (PyTorch)	~499 MB	PyTorch / GPU	Training only; not deployed
distilroberta-base (PyTorch)	~250 MB	PyTorch / GPU	Distillation checkpoint; not deployed
student model.onnx (FP32)	~250 MB	ONNX Runtime	Intermediate export; not deployed
student model-int8.onnx	~125 MB	ONNX Runtime (CPU)	Cloud Mode FastAPI + Local Mode Flask

Table 4.7: Model Artefact Sizes by Pipeline Stage

The 75 per cent size reduction from the original fine-tuned RoBERTa checkpoint (499 MB) to the deployed INT8 ONNX student (125 MB) reflects the combined effect of distillation (499 MB → ~250 MB) and INT8 quantization (~250 MB → 125 MB). ONNX Runtime's CPU execution provider handles the INT8 weight dequantization transparently, with no code changes required between development and production inference.

4.7 Inference Pipeline

The deployed inference pipeline (src/inference.py) runs entirely within ONNX Runtime on CPU, with no PyTorch or GPU dependency. Given an input text string, the pipeline proceeds as follows: (1) the text is tokenized using the saved RoBERTa tokenizer with truncation to 512 tokens and padded to maximum length; (2) the tokenized input_ids and attention_mask are passed as NumPy arrays to the ONNX Runtime InferenceSession; (3) the session runs the INT8 quantized graph and returns raw signal logits; (4) a sigmoid function converts logits to probabilities; (5) each probability is compared to its per-label optimised threshold; (6) labels whose probability exceeds their threshold are returned as active signals with their probability score. The inference function is stateless and thread-safe, enabling concurrent request handling in both Flask and FastAPI backends.

Step	Component	Output
1. Tokenization	RoBERTa tokenizer (saved with model)	input_ids [1, 512], attention_mask [1, 512] as NumPy arrays
2. ONNX inference	InferenceSession (model-int8.onnx)	signal_logits [1, 8] as NumPy array
3. Sigmoid activation	NumPy sigmoid: 1/(1+exp(-logits))	signal_probs [1, 8] in [0, 1]
4. Threshold comparison	Per-label optimised thresholds	Active signal flags [1, 8] as bool

5. Output assembly	Dict comprehension over LABELS	{ label: probability } for active labels only

Table 4.8: Deployed Inference Pipeline Steps

4.8 Ablation Study Design

The V2.0 training plan specifies a structured ablation study to quantify the contribution of each pipeline component. Five model variants are evaluated on the held-out test set and the cross-platform holdout split:

Ablation Variant	Description	Purpose
V1 RoBERTa baseline	Original single-label fine-tuned RoBERTa; proxy subreddit labels	Establishes V1 performance ceiling
Teacher without DAPT	RoBERTa-base fine-tuned directly on annotated data, no domain pre-training	Isolates DAPT contribution
Teacher with DAPT	DAPT checkpoint fine-tuned on annotated data; full teacher pipeline	Measures DAPT gain
Student (distilled)	DistilRoBERTa distilled from DAPT teacher; INT8 ONNX export	Measures distillation quality drop
Student without severity head	Distilled model without multi-task severity auxiliary loss	Isolates multi-task benefit

Table 4.9: Planned Ablation Variants

Each variant is evaluated on macro-F1 (primary metric), per-label precision and recall, AUROC for each signal class, expected calibration error (ECE), and CPU inference latency. The cross-platform holdout evaluation directly assesses whether training exclusively on Reddit posts generalises to YouTube comment and Twitter post language patterns.

CHAPTER 5: TECHNOLOGY STACK

5.1 Browser Extension and Backend Technologies

The Chrome extension is built to MV3 using vanilla HTML, CSS, and JavaScript with no build toolchain, bundling Chart.js locally to comply with MV3 content security policies. The local backend uses Flask with Gunicorn (WSGI) and Flask-CORS. The Hugging Face Space uses FastAPI with Uvicorn (ASGI) and Pydantic for request validation.

Layer	Technology	Version / Notes
Extension	Chrome Manifest V3	MV3 specification
Extension	Chart.js + Vanilla JS/HTML/CSS	Bundled local copy; no bundler
Cloud API	FastAPI + Uvicorn + Pydantic	Python 3.11
Local API	Flask + Gunicorn + Flask-CORS	Python 3.10
Inference	ONNX Runtime (CPUExecutionProvider)	Both deployment paths
Inference	Hugging Face Transformers	AutoTokenizer
Inference	ekam28/emotion-detector-onnx	INT8 quantised RoBERTa
Database	SQLite (sqlite3)	Python built-in module
Container	Docker + Docker Compose	python:3.10-slim base

Table 5.1: Technology Stack Summary

5.2 Inference Runtime, Database, and DevOps

ONNX Runtime with CPUExecutionProvider is used for inference in both deployment paths, loading the INT8-quantized RoBERTa model from ekam28/emotion-detector-onnx on Hugging Face Hub. SQLite is used via Python's built-in sqlite3 module for all local persistence. Docker and Docker Compose manage local deployment with named volumes persisting the model cache and the database across container restarts.

Tool	Purpose
GitHub Actions	CI/CD: test, lint, Docker smoke test, CodeQL, release zip
pytest	Backend unit and integration testing
Flake8 / Black	Python linting and formatting
Prettier	JavaScript / HTML / CSS formatting
CodeQL	Static security analysis (Python and JavaScript)

Table 5.2: DevOps and Quality Assurance Tools

CHAPTER 6: SYSTEM ARCHITECTURE AND DESIGN

6.1 Architectural Overview

Mental Wellbeing Guard is composed of three independently deployable components: the Chrome MV3 browser extension, the Hugging Face Space cloud API, and the Dockerized Flask local backend. The extension is the single client; the two backend components are alternative inference targets. The extension's mode manager determines which backend receives inference requests based on the user's selected mode and a health check result in Local Mode.

[Figure: Figure 6.1: Overall Dual-Mode System Architecture]

Figure 6.1: Overall Dual-Mode System Architecture

6.2 Cloud and Local Mode Data Flows

In Cloud Mode, comment text extracted by the extension is sent via HTTPS POST to the /batch endpoint of the Hugging Face Space at https://ekam28-emotion-detector-api.hf.space. ONNX Runtime loads the quantised RoBERTa model and returns JSON label probability distributions. No data is persisted in Cloud Mode.

[Figure: Figure 6.2: Cloud Mode Data Flow]

Figure 6.2: Cloud Mode Data Flow

In Local Mode, the extension performs a health check against http://localhost:8000/health before routing any inference request. If the health check succeeds, inference requests go to the Flask backend's /api/analyze endpoint, which applies the same ONNX inference pipeline, persists events to SQLite, and serves the web hub, dashboard, and settings APIs.

[Figure: Figure 6.3: Local Mode Data Flow]

Figure 6.3: Local Mode Data Flow

6.3 Extension Module Architecture

manifest.json declares all permissions and host matches for YouTube, Reddit, X, Twitter, the Hugging Face Space, and localhost:8000. background.js (the MV3 service worker) tracks tab-level session times via chrome.tabs events, discards sessions under 10 seconds, and posts valid events to /api/events in Local Mode. content.js handles manual comment extraction using platform-specific CSS selectors from config.js. page-monitor.js uses a MutationObserver with a 1,400 ms debounce to sample up to 24 comments per cycle, applying Smart Shield overlays and handling nudges, breather mode, and draft support prompts. The three-tab popup (Dashboard, Thread Analysis, Settings) uses Chart.js for visualisation and provides the Cloud/Local toggle and shield threshold slider.

[Figure: Figure 6.4: Extension Module Architecture]

Figure 6.4: Extension Module Architecture

Label Index	Label Name	Distress Weight
LABEL_0	ADHD	0.40
LABEL_1	Anxiety	0.72

LABEL_2	Autism	0.35
LABEL_3	BPD	0.88
LABEL_4	Depression	0.82
LABEL_5	PTSD	0.76
LABEL_6	Normal	0.00

Table 6.1: Label Mapping: Model Output to Display Category

6.4 Local Web Hub and SQLite Store

The local web hub (served at http://localhost:8000) provides five sections: Dashboard, Self-Check, Recent Events, Settings, and Support Resources. The SQLite store maintains two tables: events (id, timestamp, source_url, platform, label_counts JSON, risk_score, risk_band, event_type) and settings (a single-row user preferences record persisted across container restarts).

6.5 API Contracts

Method	Endpoint	Purpose
GET	/health	Return backend status and version
POST	/api/analyze	Classify comment batch; return label distributions and risk scores
GET	/api/dashboard	Return 7-day aggregated wellbeing metrics
GET	/api/events	Return paginated stored wellbeing events
POST	/api/events	Insert new wellbeing event (service worker)
POST	/api/self-check	Insert self-check response event
GET/PATCH/PUT	/api/settings	Read or update wellbeing settings

GET	/api/support-resource	Return locale-aware mental health resource

Table 6.2: Local Flask Backend API Endpoints

Method	Endpoint	Purpose
GET	/health	Return Space status, label count, and model identifier
POST	/predict	Classify a single text; return sorted label probabilities
POST	/batch	Classify up to 100 texts; return one distribution per input

Table 6.3: Hugging Face Space API Endpoints

6.6 Risk Scoring and Wellbeing Metrics

The risk score is computed deterministically from the ONNX classifier output (not a model output, not learned from data):

risk = clamp((1 - normal_score) x 0.55 + weighted_distress x 0.45 + keyword_boost)

normal_score is the softmax probability for LABEL_6 (Normal). weighted_distress is the dot product of label probabilities and distress weights from Table 5.1. keyword_boost adds up to 0.24 when phrases such as 'suicidal' or 'self harm' are detected. The result is clamped to [0, 1] and assigned to a risk band:

Band	Score Range	Description	Product Action
calm	0.00 - 0.39	Predominantly normal language	No intervention
guarded	0.40 - 0.67	Mixed or mildly elevated signal	Optional monitoring

high	0.68 - 0.83	Elevated distress-weighted signal	Shield if threshold met; dashboard flag
extreme	0.84 - 1.00	Strongly elevated multi-label signal	Shield applied; rabbit-hole nudge eligible

Table 6.4: Risk Score Thresholds and Bands

The backend also computes session volatility (flagged when max risk swing between consecutive events exceeds 0.45, or average swing exceeds 0.30) and an emotional-diet breakdown (label exposure percentages over the 7-day dashboard window).

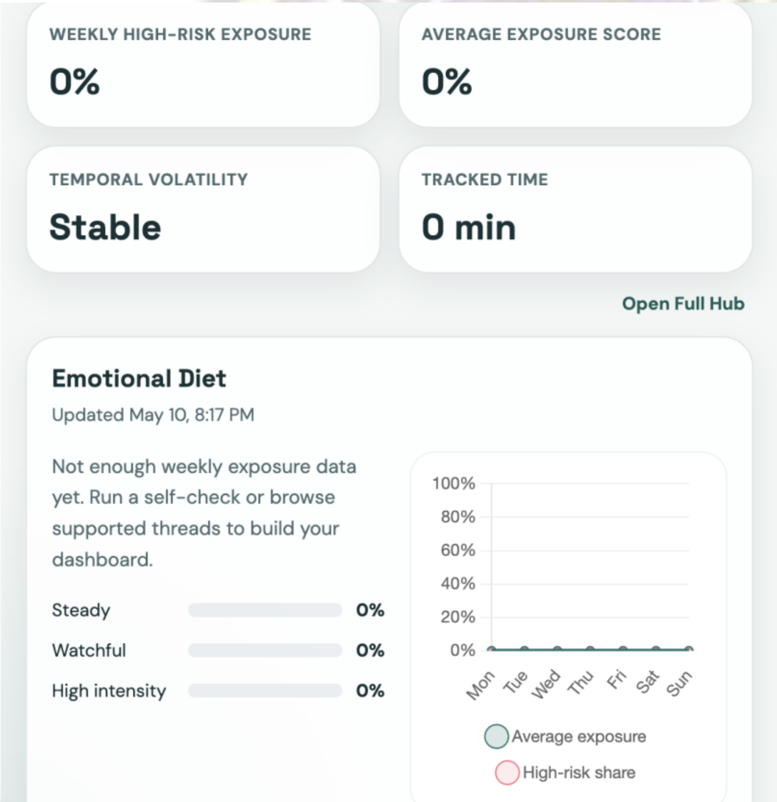

Figure: Emotional Diet

6.7 Key Implementation Constants

Constant	Value	Purpose
Max comments per API request	100	Protects backend from oversized payloads

Extension manual batch size	30	Keeps popup analysis responsive
Passive auto-analysis sample size	24	Limits background network cost per cycle
Max token length	512	Tokenizer truncation limit
Auto-analysis debounce	1,400 ms	Avoids redundant calls during scroll
Local API timeout	60 s	Sufficient for local ONNX inference
Cloud API timeout	120 s	Allows HF Space cold-start to complete
Minimum tracked session	10 s	Filters out very short accidental sessions
Dashboard window	7 days	Weekly overview for the dashboard
Default shield threshold	0.62	Risk score above which Smart Shield is applied

Table 6.5: Key Implementation Constants

Artefact	**Repository**	**Storage Size**	**Notes**
Original fine-tuned model	ekam28/emotion-detector	~499 MB	Full-precision FP32; PyTorch format
Quantised ONNX artefact	ekam28/emotion-detector-onnx	~125 MB	INT8 quantised; served by ONNX Runtime

Table 6.6: Model Size Comparison: Original vs. Quantized ONNX

CHAPTER 7: REQUIREMENTS

The following table consolidates all functional (FR) and non-functional (NFR) requirements and their implementation status.

Req. ID	Category	Description	Status
FR-01 to FR-06	Extraction	Comment extraction from four platforms; batching, sampling, deduplication, truncation	Implemented
FR-07 to FR-11	Mode Mgmt	Cloud/Local toggle; health-check routing; cold-start retry with exponential back-off	Implemented
FR-12 to FR-15	Inference	Seven-label softmax distribution; top-k output; deterministic risk score; four risk bands	Implemented
FR-16 to FR-19	Extension UI	Dashboard, Thread Analysis, Settings tabs; Chart.js doughnut; label filter	Implemented
FR-20 to FR-25	Interventions	Smart Shield (click-to-reveal); rabbit-hole nudge; breather mode; draft prompt; locale resources	Implemented
FR-26 to FR-30	Local Backend	Web hub; SQLite persistence; 7-day dashboard; self-check endpoint; support resource endpoint	Implemented
NFR-01 to NFR-03	Privacy	Local Mode data isolation; Cloud Mode disclosure; no PII collection	Implemented
NFR-04 to NFR-07	Performance	1,400 ms debounce; 60 s / 120 s timeouts; 10 s session minimum	Implemented

NFR-08 to NFR-10	Usability	Zero-setup Cloud Mode; no-auth web hub; dismissable interventions	Implemented
NFR-11 to NFR-13	Explainability	Probability outputs; deterministic risk score; non-medical copy	Implemented
NFR-14 to NFR-16	Safety	No clinical claims; verified helplines; non-destructive overlays	Implemented
NFR-17 to NFR-20	Maintainability	CI: Flake8, Black, Prettier, CodeQL, pytest enforced on every push	Implemented

Requirements Summary

CHAPTER 8: IMPLEMENTATION DETAILS

8.1 Extension Implementation

manifest.json declares host permissions for YouTube, Reddit, X, Twitter, the Hugging Face Space, and localhost:8000. The MV3 service worker (background.js) tracks tab session times via chrome.tabs events, discards sessions under 10 s, and posts valid events to /api/events in Local Mode. Comment extraction relies on platform-specific CSS selectors in config.js: YouTube targets ytd-comment-thread-renderer; Reddit targets shreddit-comment; X and Twitter target article[data-testid].

page-monitor.js uses a MutationObserver with a 1,400 ms debounce, sampling up to 24 visible comments per cycle and skipping already-processed nodes tracked in a session-level Set. Manual analysis divides extracted comments into batches of 30 and sends them to the /batch (HF Space) or /api/analyze (local) endpoint. The Cloud/Local mode manager stores the current mode in chrome.storage.local; a health check to /health precedes every Local Mode inference call, with automatic fallback to Cloud Mode and a Settings-tab warning on failure.

The Smart Shield applies a CSS blur overlay to comments scoring at or above the configured threshold (default 0.62); clicking the overlay removes the blur. The rabbit-hole nudge is a fixed-position injected div triggered when extended session time intersects a rolling average risk above 0.40. Breather mode applies a full-page shield with a 5-minute countdown, dismissable early. The draft support prompt watches input events on textarea and contenteditable elements; on keyword match, a locale-aware helpline panel is injected adjacent to the field without intercepting typed content.

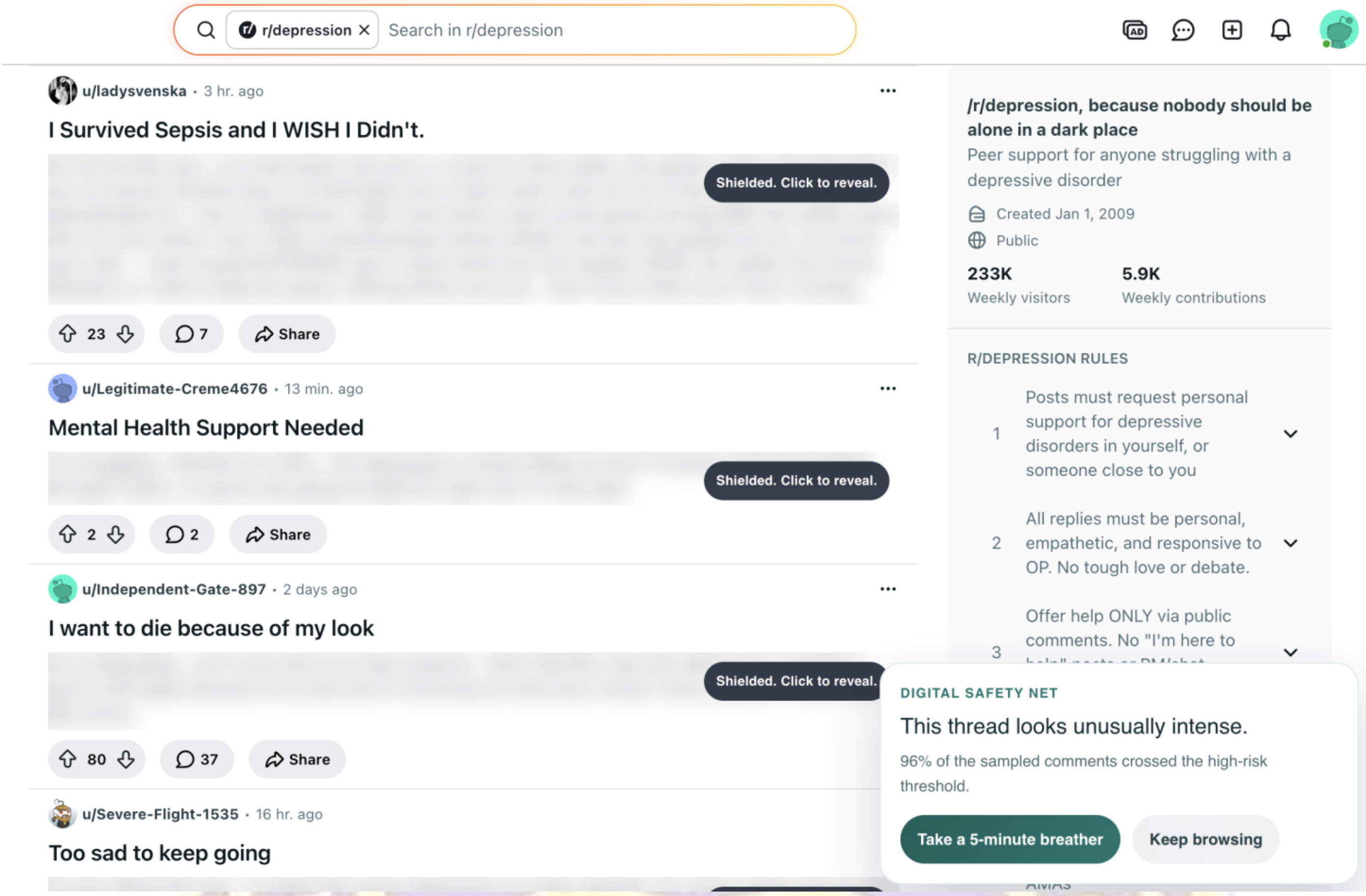

Figure 8.1: Smart Shield and Wellbeing Interventions Flow

8.2 Hugging Face Space API

The Space is hosted at https://ekam28-emotion-detector-api.hf.space as a Docker SDK Space (python:3.11-slim, Uvicorn on port 7860, CORS enabled). On startup, snapshot_download fetches the ONNX artifact from ekam28/emotion-detector-onnx pinned to a specific revision hash. ONNX Runtime InferenceSession is initialised with CPUExecutionProvider; AutoTokenizer is loaded from the same repository. /predict classifies a single text; /batch classifies up to 100. Both apply softmax over raw logits and return sorted label-probability pairs. Pydantic models enforce a 5,000 character maximum per text and a 100-item batch limit.

8.3 Local Flask Backend

The Flask app factory (app/__init__.py) reads config from environment variables, registers Flask-CORS and the API Blueprint, and initialises the SQLite store on startup. The inference module (inference.py) loads the ONNX artifact from the Docker volume, tokenizes with max 512 tokens, runs the ONNX session, applies softmax, and returns sorted label-probability dicts. The store module (store.py) creates the events and settings tables on first run and provides all CRUD functions. The wellbeing module (wellbeing.py) implements the risk score formula, volatility detection, and emotional-diet aggregation. Routes (routes.py) registers all API and web hub endpoints on a Blueprint with input validation returning HTTP 400 for invalid payloads.

8.4 Docker Deployment

docker-compose.yml defines a single backend service exposing port 8000 with two named volumes: wellbeing-data at /app/data (SQLite database) and wellbeing-model-cache at /home/appuser/model-cache (ONNX artifact). The Dockerfile uses python:3.10-slim with a non-root appuser. bootstrap_model.py downloads the ONNX artifact from Hugging Face Hub on first run (pinned to a specific commit hash) and skips the download if the artifact already exists in the volume. The Compose health check polls /health every 30 seconds.

8.5 Implemented Features Summary

Component	Feature	Implementation File
Extension	MV3 service worker + session tracking	manifest.json, background.js
Extension	Platform selectors (YouTube/Reddit/X/Twitter)	config.js
Extension	Passive monitoring with debounce	page-monitor.js
Extension	Manual batch analysis (batch size 30)	js/ popup modules

Extension	Cloud / Local mode toggle + health check	config.js, js/ mode manager
Extension	Smart Shield with click-to-reveal	page-monitor.js, content.css
Extension	Rabbit-hole nudge	page-monitor.js, content.css
Extension	Five-minute breather mode	page-monitor.js, content.css
Extension	Draft support prompts + locale resources	page-monitor.js, config.js
Extension	Chart.js popup dashboard (3-tab UI)	popup.html, js/
HF Space	FastAPI + Uvicorn + ONNX Runtime	hf-space/app.py
HF Space	/predict and /batch endpoints	hf-space/app.py
Local Backend	Flask app factory + Flask-CORS	app/__init__.py, config.py
Local Backend	ONNX Runtime inference + tokenizer	app/inference.py
Local Backend	SQLite wellbeing store	app/store.py
Local Backend	Risk scoring + volatility + diet	app/wellbeing.py
Local Backend	Full REST API (8 endpoints)	app/routes.py
Local Backend	Server-rendered web hub	templates/index.html, app.js
DevOps	Docker Compose + health check	docker-compose.yml, Dockerfile

DevOps	Pytest backend test suite	backend/tests/
DevOps	GitHub Actions CI (test, lint, Docker, CodeQL, release)	.github/workflows/

Table 8.1: Implemented Features Checklist

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Historical Model Evaluation (V1 Phase)

The V1 evaluation used a held-out test set from the fine-tuning dataset (community-derived proxy labels, not clinician-verified). Macro F1 is reported because the label distribution is frequently imbalanced in mental health social media datasets.

Model	Accuracy (%)	Macro F1	Notes
Fine-tuned RoBERTa	85.69	0.8601	Best V1 model; used in production
BERT-base (fine-tuned)	Lower than RoBERTa	Lower than RoBERTa	Transformer baseline
SVM + TF-IDF	Lower than transformers	Lower than transformers	Traditional ML baseline
Lexicon rule-based	Lowest	Lowest	Rule-based comparison

Table 9.1: Historical V1 Model Performance Comparison

9.2 ONNX Quantisation Results

Artefact	Repository	Approx. Size	Reduction
Original fine-tuned model	ekam28/emotion-detector	~499 MB	-
Quantised ONNX artefact	ekam28/emotion-detector-onnx	~125 MB	~75%

Table 9.2: Model Size Comparison — Original vs. Quantised ONNX

The 75 per cent storage reduction reduces HF Space cold-start time, lowers Docker volume requirements, and improves per-request inference latency on the local backend.

9.3 V1.0 to V2.0 Product Improvements

Dimension	V1.0	V2.0
Inference runtime	PyTorch local backend	ONNX Runtime (local and cloud)
Deployment modes	Local only	Cloud Mode (default) + Local Mode
Setup for new users	Requires Docker	Zero-setup in Cloud Mode
Privacy option	No explicit choice	Local Mode: fully local inference and storage
Web hub	Not available	Local Flask web hub with dashboard and self-check
History and trends	Not available	SQLite wellbeing store; 7-day dashboard window
Content safety UX	Comment labels only	Smart Shield, nudge, breather, draft prompts
Extension UI	Single popup view	Three-tab popup: Dashboard, Thread, Settings
Risk communication	Label name and probability	Risk score, risk band, and label distribution
Ethical framing	Disorder detection	Signal analysis and digital wellbeing
CI/CD	No CI	GitHub Actions: test, lint, Docker, CodeQL, release

Table 9.3: V1.0 to V2.0 Product Improvements

9.4 Discussion

The distribution-aware risk score is preferred over top-label output because a comment with 55 per cent Normal / 45 per cent Depression represents a meaningfully different wellbeing signal than 55 per cent Normal / 45 per cent ADHD — even though both share the same top label. The distress-weight formula captures this distinction. The V1 accuracy figure of 85.69 per cent should be understood as classifier agreement with proxy dataset labels, not as diagnostic accuracy for clinical conditions. Manual validation of the deployed system confirms consistent ONNX inference: emotionally distressed text consistently scores high on Depression or BPD with correspondingly high risk scores; neutral text scores high on Normal with low risk scores.

CHAPTER 10: ETHICAL CONSIDERATIONS AND PRIVACY

10.1 Signal Analysis vs. Clinical Diagnosis

Mental Wellbeing Guard is not a clinical system. It does not diagnose mental illness, classify users as patients, or assess individual risk in a clinical sense. These are deliberate design constraints, not limitations to overcome. A text classifier trained on proxy-labelled social media data cannot replicate the structured assessment process of a qualified mental health professional. All outputs are framed as probabilistic language signals and wellbeing metrics throughout the system.

10.2 Privacy Architecture

In Cloud Mode, comment text is transmitted to the stateless Hugging Face Space, which does not store request payloads. This behaviour is disclosed in the Settings tab and extension description. In Local Mode, all inference, storage, and dashboard computation occurs on the user's own machine — no comment text leaves the device. The SQLite database is stored in a Docker volume the user controls and can delete to remove all stored history. Neither mode collects personally identifiable information; the system analyses comment text, not user account data or credentials.

10.3 Data Ethics, User Autonomy, and Responsible AI Principles

The classifier was fine-tuned on proxy-labelled data (community membership rather than clinician-verified labels), which may over-represent users who publicly self-identify with a condition and may not transfer equally across all demographics and languages. All content shielding interventions are reversible overlays: shielded comments are click-to-reveal; breather mode is dismissable early; nudges are dismissable without action; draft prompts do not intercept typed content. Support resources are links to verified mental health helplines (including India-specific resources such as iCall and Vandrevala Foundation), not AI-generated advice.

Six responsible AI principles were applied throughout: non-diagnostic framing; transparency (deterministic risk score formula, mode-dependent privacy disclosure in UI); user control (all interventions dismissable, threshold and mode configurable by the user); privacy by design (stateless cloud path separated from stateful local path); no profiling (feed-level session signals only, no persistent user mental health profile); and harm avoidance (overlay-based, non-destructive interventions that do not restrict user access to any content).

CHAPTER 11:TESTING AND VALIDATION

11.1 Backend Automated Tests

The backend test suite is implemented using pytest against a temporary in-memory SQLite database. Table 10.1 lists the 18 implemented test cases.

Test ID	Endpoint / Module	Test Description
T-B-01	GET /health	Returns HTTP 200 with status ok and correct version string
T-B-02	POST /api/analyze	Returns HTTP 400 for empty comment list
T-B-03	POST /api/analyze	Returns HTTP 400 for comment exceeding 5,000 character limit
T-B-04	POST /api/analyze	Returns HTTP 400 for list exceeding 100 comment limit
T-B-05	POST /api/analyze	Returns HTTP 200 with correct label keys for valid input
T-B-06	inference.py	Top-k filter returns at most k labels per comment
T-B-07	inference.py	Softmax output sums to approximately 1.0 per input
T-B-08	GET /api/settings	Returns HTTP 200 with default settings for new installation
T-B-09	PATCH /api/settings	Returns HTTP 200 and updates shield threshold correctly
T-B-10	PUT /api/settings	Replaces full settings object and returns updated state
T-B-11	GET /api/dashboard	Returns HTTP 200 with empty stats for store with no events

T-B-12	GET /api/dashboard	Returns correct risk band distribution for seeded event data
T-B-13	POST /api/events	Returns HTTP 201 for valid event insertion
T-B-14	POST /api/events	Returns HTTP 400 for event with missing required fields
T-B-15	POST /api/self-check	Returns HTTP 201 for valid mood and note submission
T-B-16	GET /api/support-resource	Returns India resource for IN locale query parameter
T-B-17	GET /api/support-resource	Returns fallback resource for unknown locale
T-B-18	config.json	Label mapping contains exactly seven entries

Table 11.1: Backend Automated Test Cases

11.2 CI Workflow Validation

Workflow	Trigger	Validation Performed
test	Push / PR to main	Pytest backend suite; fails build on test failure
lint	Push / PR to main	Flake8 + Black (Python); Prettier (JS/HTML/CSS)
docker-build	Push / PR to main	Docker Compose build and smoke test against /health
codeql	Push / PR / schedule	CodeQL static analysis for Python and JavaScript
release	Tag push (v*)	Packages extension into downloadable release zip

Table 11.2: GitHub Actions CI Workflows

11.3 Manual Validation Summary

Manual validation was performed across all three system components. Extension: comment extraction confirmed on all four platforms; Smart Shield blurring and click-to-reveal verified; Cloud/Local mode switching and fallback to Cloud Mode on backend unavailability confirmed. HF Space: health, /predict, and /batch endpoints verified, including HTTP 422 for oversized batch requests. Local web hub: dashboard loading, self-check submission, settings persistence across container restarts, and locale-aware support resource display confirmed. Wellbeing interventions: Smart Shield threshold behaviour, breather mode countdown and early dismissal, and draft support prompt injection without content interception all verified.

CHAPTER 12: LIMITATIONS

This chapter presents the known limitations of Mental Wellbeing Guard v2.0. Acknowledging limitations is essential for responsible reporting of an AI system operating in a sensitive domain.

Category	Limitation	Impact
Model / Data	Proxy label bias: labels from community membership, not clinician-verified diagnoses	May not generalise across all user populations
Model / Data	Single-label softmax forces probability mass to sum to one; co-occurring signals cannot be expressed	Cannot model simultaneous multi-condition signals
Model / Data	No severity modelling: only label category predicted, not signal intensity	High and low intensity comments receive the same label
Model / Data	English-only model: not designed for code-mixed (Hinglish) or Indian regional languages	Significant gap for the primary Indian user audience
Architecture	Dashboard history unavailable in Cloud Mode (stateless cloud path)	Cloud Mode users cannot access wellbeing trends
Architecture	HF Space cold-start latency: 30-90 s on free cpu-basic tier after inactivity	Degraded first-request UX in Cloud Mode
Architecture	No mobile browser support: Chrome MV3 extensions not available on mobile Chrome or Safari iOS	Mobile social media use cases not addressable

Ethical	Uncalibrated softmax probabilities: no temperature scaling applied	Risk score is a relative ranking tool, not absolute measure
Ethical	No fairness evaluation across demographic subgroups, languages, or regions	Systematic performance disparities cannot be ruled out
Ethical	Draft prompt cannot distinguish personal distress from discussion or creative writing	May surface resources in irrelevant contexts
Platform	CSS selector fragility: platform DOM updates break extraction until selectors are updated	Ongoing maintenance required
Platform	Only four platforms supported; Instagram, Facebook, LinkedIn not covered	Large portion of social media excluded

Table 12.1: Limitations Summary

CHAPTER 13: FUTURE SCOPE

The following items represent planned improvements and extensions beyond the current v2.0 implementation. None have been implemented in the current submission.

Area	Future Work Item	Priority
Model	True multi-label classifier with sigmoid output	High
Model	Supervised ordinal severity estimation	High
Model	Browser-native WebGPU / WebNN inference (no backend required)	Medium
Model	Calibration via temperature scaling; uncertainty display in UI	Medium
Language	Multilingual and code-mixed Indian language support (MuRIL / XLM-RoBERTa)	High
Privacy	User-controlled export and deletion of local wellbeing history	High
Privacy	Differential privacy / federated learning for future model improvement	Low
Platform	Instagram, Facebook, LinkedIn, and Indian platform adapters	Medium
Platform	Mobile application (Android / iOS) using same cloud API	Medium
Ethics	Clinician-reviewed safety copy and resource links	High
Ethics	Fairness audit and bias mitigation across demographic subgroups	High

Table 13.1: Future Scope Summary

CHAPTER 14: CONCLUSION

14.1 Project Summary

Mental Wellbeing Guard is a dual-mode Chrome extension and local web hub that analyses mental-health-related language signals in social media comments. The final v2.0 system achieves four goals the original prototype did not: zero-setup Cloud Mode via Hugging Face Spaces; a privacy-first Local Mode with fully local inference and history storage; an expanded product platform with a risk scoring engine, interactive dashboard, self-check workflow, and four content-safety interventions; and replacement of diagnostic language with responsible signal-analysis framing throughout.

14.2 Technical and Responsible AI Contributions

Four technical contributions are made: (1) a dual-mode inference architecture with transparent health-check-based switching; (2) ONNX Runtime deployment achieving a 75 per cent model storage reduction with portability across Flask and FastAPI without framework-specific serving dependencies; (3) a distribution-aware risk scoring engine using label-specific distress weights; and (4) integration of wellbeing-oriented UX features (Smart Shield, nudge, breather mode, draft support) into a browser extension as a practical application of responsible AI design.

Beyond the technical system, the project demonstrates that responsible framing and engineering quality are not in tension: the v2.0 system is more sophisticated than the v1 prototype in every dimension while applying stricter ethical constraints at every level. Mental Wellbeing Guard addresses a real problem — the cumulative emotional impact of high-distress social media content — with a practical, privacy-aware tool grounded in a fine-tuned RoBERTa classifier, served through ONNX Runtime, and delivered through a usable browser extension. It surfaces patterns, prompts reflection, and connects users to verified resources. It does not diagnose, treat, or replace professional mental health support.

REFERENCES

[1] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS, 30, 5998-6008.

[2] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT, 4171-4186.

[3] Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

[4] Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT. NeurIPS 2019 Workshop. arXiv:1910.01108.

[5] Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531.

[6] Coppersmith, G., Dredze, M., and Harman, C. (2014). Quantifying Mental Health Signals in Twitter. ACL CLPsych, 51-60.

[7] Tadesse, M. M. et al. (2019). Detection of Depression-Related Posts in Reddit Social Media Forum. IEEE Access, 7, 44883-44893.

[8] Ratner, A. et al. (2017). Snorkel: Rapid Training Data Creation with Weak Supervision. VLDB Endowment, 11(3), 269-282.

[9] Gkotsis, G. et al. (2017). Characterisation of Mental Health Conditions in Social Media using Informed Deep Learning. Scientific Reports, 7, 45141.

[10] Losada, D. E., and Crestani, F. (2016). A Test Collection for Research on Depression and Language Use. CLEF, LNCS 9822, 28-39.

[11] Ji, S. et al. (2021). Suicidal Ideation Detection: A Review of Machine Learning Methods and Applications. IEEE Trans. Computational Social Systems, 8(1), 214-226.

[12] Ive, J. et al. (2018). Hierarchical Neural Model with Attention for Classification of Social Media Text Related to Mental Health. CLPsych, 69-77.

[13] Microsoft Corporation. (2024). ONNX Runtime Documentation. https://onnxruntime.ai/docs/. Accessed: May 2026.

[14] Hugging Face Inc. (2024). Hub and Spaces Documentation. https://huggingface.co/docs/. Accessed: May 2026.

[15] Google Chrome Developers. (2024). Chrome Extensions Manifest V3 Overview. https://developer.chrome.com/docs/extensions/mv3/. Accessed: May 2026.

[16] Pallets Projects. (2024). Flask Documentation. https://flask.palletsprojects.com/. Accessed: May 2026.

[17] FastAPI. (2024). FastAPI Documentation. https://fastapi.tiangolo.com/. Accessed: May 2026.

[18] Docker Inc. (2024). Docker Documentation. https://docs.docker.com/. Accessed: May 2026.

[19] Wolf, T. et al. (2020). Transformers: State-of-the-Art NLP. EMNLP System Demonstrations, 38-45.

[20] Bittman, E. et al. (2026). ekam28/emotion-detector-onnx. Hugging Face. https://huggingface.co/ekam28/emotion-detector-onnx. Accessed: May 2026.

[21] Bittman, E. et al. (2026). ekam28/emotion-detector-api. Hugging Face Space. https://huggingface.co/spaces/ekam28/emotion-detector-api. Accessed: May 2026.

[22] Conway, M., and O'Connor, D. (2016). Social Media, Big Data, and Mental Health. Current Opinion in Psychology, 9, 77-82.

[23] Chancellor, S., and De Choudhury, M. (2020). Methods in Predictive Techniques for Mental Health Status on Social Media. NPJ Digital Medicine, 3(1), 1-11.

[24] Bender, E. M. et al. (2021). On the Dangers of Stochastic Parrots. FAccT, 610-623.